ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

LAB 02

NHẬP MÔN KHOA HỌC DỮ LIỆU

| Thành viên |

Bùi Trung Hiếu (23120257)

Ngành: Nhóm ngành Máy tính và Công nghệ thông tin

Giáo viên hướng dẫn: Huỳnh Lâm Hải Đăng

Thành phố Hồ Chí Minh – 2025

1. Mục lục

1. Mục lục 2
2. Giới thiệu chung 4
3. Phân tích cấu trúc phân cấp và chuẩn hóa nội dung..... 4
3.1. Thu thập đa tệp 5
3.1.1. Xác định tệp LaTeX chính 5
3.1.2. Làm phẳng nội dung 5
3.1.3. Các chỉ số thống kê..... 6
3.2. Xây dựng cấu trúc phân cấp nội dung 6
3.2.1. Ý tưởng..... 6
3.2.2. Các chỉ số thống kê..... 9
3.3. Chuẩn hóa và loại bỏ trùng lặp dữ liệu..... 10
3.3.1. Chuẩn hóa dữ liệu và trích xuất tài liệu tham khảo 10
3.3.2. Loại bỏ trùng lặp 12
3.3.3. Các chỉ số thống kê..... 14
4. Quy trình so khớp tài liệu tham khảo..... 14
4.1. Chọn dữ liệu 14
4.2. Tiền xử lý dữ liệu 15

4.2.1. Tiêu đề 15
4.2.2. Tác giả 16
4.2.3. Năm xuất bản 17
4.2.4. Nơi xuất bản 18
4.3. Gán nhãn dữ liệu 19
4.3.1. Ý tưởng 19
4.3.2. Các chỉ số thống kê 21
4.4. Xây dựng đặc trưng 22
4.4.1. Các đặc trưng liên quan đến tiêu đề 22
4.4.2 Các đặc trưng liên quan đến tác giả 27
4.4.3. Các đặc trưng liên quan đến năm xuất bản 30
4.4.4. Đặc trưng tổng hợp: 32
4.5. Mô hình hóa dữ liệu 34
4.5.1. Dữ liệu đầu vào 34
4.5.2. Mô hình sử dụng 35
4.5.3. Mô tả chi tiết mô hình 36
4.5.4. Các bước huấn luyện mô hình 37
4.6. Đánh giá mô hình 38
5. Link Youtube 39
6. Tài liệu tham khảo 39

2. Giới thiệu chung

Dữ liệu các bài báo khoa học tồn tại chủ yếu dưới dạng phi cấu trúc được phân tán trên nhiều tệp và có thể tồn tại nhiều phiên bản khác nhau của cùng một bài báo. Ngoài ra, các mục tài liệu tham khảo chưa được chuẩn hóa và dữ liệu metadata từ Semantic Scholar, mặc dù có cấu trúc rõ ràng hơn, vẫn tồn tại sự không nhất quán so với BibTeX. Do đó, đồ án tập trung vào hai nhiệm vụ chính:

- Phân tích và chuẩn hóa mã nguồn LaTeX của các bài báo khoa học nhằm xây dựng cấu trúc phân cấp biểu diễn nội dung bài báo một cách có tổ chức.
- Xây dựng pipeline so khớp tài liệu tham khảo, trong đó các mục tham khảo được trích xuất từ LaTeX dưới dạng BibTeX sẽ được liên kết với các bản ghi metadata tương ứng của các bài báo được thu thập từ Semantic Scholar.

3. Phân tích cấu trúc phân cấp và chuẩn hóa nội dung

Mục tiêu của quá trình phân tích phân cấp và chuẩn hóa nội dung là chuyển đổi mã nguồn LaTeX thô của bài báo khoa học thành một dạng dữ liệu có cấu trúc, nhất quán và phản ánh đúng nội dung xuất hiện trong tài liệu PDF cuối cùng.

3.1. Thu thập đa tệp

3.1.1. Xác định tệp LaTeX chính

- Một tệp .tex được coi là tệp chính nếu nó chứa đầy đủ cấu trúc của một tài liệu LaTeX hoàn chỉnh, cụ thể là có cả hai lệnh ***\begin{document}*** và ***\end{document}*** (sau khi loại bỏ các dòng comment). Dựa trên tiêu chí này, hệ thống tự động phát hiện tất cả các tệp main trong mỗi phiên bản của bài báo.
- Từ mỗi tệp chính đã được xác định, hệ thống tiến hành duyệt đệ quy theo chiều sâu để lần theo toàn bộ chuỗi các lệnh chèn tệp như ***\input, \include, \subfile, \import, \subimport và \inputfrom***. Mỗi lệnh chèn được ánh xạ tới tệp .tex tương ứng và được đưa vào danh sách các tệp được sử dụng.
- Trường hợp đặc biệt của lệnh ***\includeonly*** cũng được xử lý theo đúng ngữ nghĩa của LaTeX. Khi ***\includeonly*** được khai báo, chỉ các tệp được liệt kê trong lệnh này mới được phép xuất hiện thông qua ***\include***. Ví dụ, nếu có ***\includeonly{a,b,c}*** thì chỉ các file a, b, c mới được include (lệnh ***\include*** các file khác bị bỏ qua). Mặc khác, các file được trích từ lệnh ***\input*** và các biến thể khác không bị ảnh hưởng bởi ***\includeonly***.

3.1.2. Làm phẳng nội dung

- Sau khi xác định được tập tệp LaTeX hợp lệ, hệ thống tiến hành bước chèn trực tiếp toàn bộ nội dung của các tệp con được vào tệp chính. Cụ thể, mỗi lệnh ***\input{file}***

hoặc các biến thể tương đương sẽ được thay thế bằng nội dung thực tế của tệp được tham chiếu. Quá trình này được thực hiện đệ quy, do các tệp con cũng có thể chứa các lệnh chèn khác, cho đến khi không còn lệnh include hay input nào tồn tại trong tài liệu.

- Toàn bộ comment LaTeX, bao gồm comment theo dòng, môi trường comment, cũng như các khối điều kiện vô hiệu như ***\iffalse ... \fi***, đều được loại bỏ trước khi thực hiện phân tích. Sau khi nội dung đã được làm phẳng, lệnh ***\includeonly*** không còn ý nghĩa và cũng được loại bỏ khỏi tài liệu.
- Kết quả cuối cùng là mỗi phiên bản của bài báo chỉ còn lại các tệp LaTeX sẽ được biên dịch thành file PDF cuối cùng. Các tệp .tex khác không còn cần thiết sẽ bị loại bỏ.

3.1.3. Các chỉ số thống kê

- Bài báo không có file .tex: 458
- Bài báo có nhiều hơn 1 file main: 1996
- Bài báo có đúng 1 file main: 2546
- Trung bình số file được input vào mỗi file main tex: 0.77

3.2. Xây dựng cấu trúc phân cấp nội dung

3.2.1. Ý tưởng

- Trong quá trình phân tích nội dung chứa trong ***\begin{document}*** và ***\end{document}*** , với nút gốc là **document,** hệ thống nhận diện các lệnh phân chia mục của LaTeX và xây dựng cấu trúc phân cấp dựa trên bảy

cấp độ từ cao đến thấp, bao gồm **part, chapter, section, subsection, subsubsection, paragraph và subparagraph**. Mỗi khi một lệnh phân chia mục được phát hiện, parser xác định vị trí của mục đó trong cây phân cấp bằng cách so sánh cấp độ của nó với các mục trước đó, từ đó thiết lập quan hệ cha–con một cách chính xác. Cụ thể, với mỗi mục tìm được, parser sử dụng stack để xác định quan hệ cha-con. Khi gặp một mục mới, parser so sánh cấp độ của nó với mục đang ở đỉnh stack. Nếu cấp độ mới cao hơn hoặc bằng, parser pop stack cho đến khi tìm được mục có cấp độ thấp hơn - đó chính là nút cha. Nếu cấp độ mới thấp hơn, mục hiện tại trở thành con của mục ở đỉnh stack.

- Nội dung giữa các mục được phân tách theo thứ tự: parser tìm các environments, đánh dấu vị trí, và xử lý phần text còn lại thành các phần tử nhỏ hơn. Mỗi environment trở thành con của mục chứa nó. Các environment này được chia thành hai nhóm:
    - Nhóm thứ nhất là các environment dạng container, có khả năng chứa các thành phần con và được phân tích đệ quy. Nhóm này bao gồm các environment như **theorem, lemma, proposition, corollary, definition, example, remark, note, proof, claim, conjecture, axiom, quote**. Các environment này được biểu diễn như các nút trung gian trong cây phân cấp, cho phép tiếp tục xây dựng cấu trúc con bên trong.

  - Nhóm thứ hai là các environment dạng nút lá, bao gồm **verbatim, lstlisting, algorithm, figure và table**. Đối với các environment này, toàn bộ nội dung bên trong được giữ nguyên và được xem như một đơn vị không thể tách nhỏ.
- Sau khi xây dựng cấu trúc phân cấp ở mức khối, hệ thống tiếp tục phân tích nội dung văn bản chi tiết hơn thông qua bước tách câu. Các câu được xác định dựa trên dấu chấm theo sau bởi khoảng trắng, đồng thời hệ thống có cơ chế nhận biết các trường hợp viết tắt phổ biến như **"i.e.", "e.g.", "etc.", "Fig.", "Eq.", "Ref.", "Vol."** nhằm tránh việc tách câu sai. Ngoài ra, việc tách câu không được thực hiện bên trong các khối công thức toán học như **$...$, \[…\]** hoặc môi trường **equation**, đảm bảo tính toàn vẹn của các biểu thức toán học.
- Bên cạnh đó, hệ thống xử lý các cấu trúc danh sách trong LaTeX, bao gồm các môi trường **itemize, enumerate, description, alphlist và romlist**. Mỗi lệnh ***\item*** được chuyển thành một nút riêng trong cây phân cấp, và nội dung bên trong mỗi item tiếp tục được phân tích đệ quy lồng cho đến khi tìm được nút lá.
- Phần tài liệu tham khảo được loại bỏ bằng cách phát hiện và bỏ qua các môi trường như **thebibliography** hoặc các lệnh ***\bibliography{}*** và **\printbibliography.**
- Phần phụ lục thường được đánh dấu bằng lệnh ***\appendix***. Khi parser gặp lệnh này, nó tạo một node Appendix là con trực tiếp của Document. Sau đó các mục nội dung tiếp

theo vẫn sử dụng cùng cú pháp phân chia như \section, \subsection, và các cấp thấp hơn tương tự như cách phân tích cấu trúc phân cấp trên.

- Hệ thống phát hiện phần lời cảm ơn bằng các lệnh độc lập như ***\acknowledgements*** hoặc ***\acks***, các section có tiêu đề chứa từ khóa "acknowledgement" (bao gồm cả dạng **\section{}** và ***\section*{}***), cũng như các environment chuyên biệt dưới dạng ***\begin{acknowledgements} ... \end{acknowledgements}***. Khi phần Acknowledgements được phát hiện, parser tạo một nút Acknowledgements làm con trực tiếp của nút Document trong cấu trúc phân cấp. Nội dung bên trong phần này sau đó được phân tích theo cùng một cơ chế như các mục thông thường, bao gồm việc tách câu, xử lý công thức toán học và nhận diện các phần tử nút lá.
- Thông qua chuỗi xử lý trên, mã nguồn LaTeX thô được chuyển đổi thành một cấu trúc cây phân cấp đầy đủ và nhất quán, phản ánh chính xác tổ chức nội dung của bài báo khoa học.

3.2.2. Các chỉ số thống kê

- Tỉ lệ parse thành công: 90.9%
- Trung bình nodes/paper: 590.7
- sent: 2039538 (trung bình 448.5/paper)
- formula: 247514 (trung bình 54.4/paper)
- item: 77848 (trung bình 17.1/paper)
- figure: 70255 (trung bình 15.5/paper)

- subsection: 63780 (trung bình 14.0/paper)
- section: 52711 (trung bình 11.6/paper)
- subsubsection: 18396 (trung bình 4.0/paper)
- paragraph: 13348 (trung bình 2.9/paper)
- chapter: 214 (trung bình 0.05/paper)
- subparagraph: 133 (trung bình 0.03/paper)

3.3. Chuẩn hóa và loại bỏ trùng lặp dữ liệu

3.3.1. Chuẩn hóa dữ liệu và trích xuất tài liệu tham khảo

- Các lệnh liên quan đến bố cục và trình bày như ***\centering, \raggedright***, các lệnh điều chỉnh khoảng cách như ***\vspace, \hspace, \medskip***, các lệnh thay đổi kích thước phông chữ như ***\small, \large, \footnotesize***, các lệnh ngắt trang như ***\clearpage, \newpage, \pagebreak***, cũng như các tùy chọn định vị như ***[htbp]*** đều được loại bỏ khỏi nội dung. Đối với các lệnh định dạng có chứa văn bản bên trong, chẳng hạn như ***\textbf{}, \textit{}*** hoặc các biến thể tương tự, parser thực hiện thao tác *unwrap*, giữ lại nội dung văn bản thuần và loại bỏ phần định dạng bên ngoài.

- Khi một phần tử được xác định là thuộc ngữ cảnh bảng, các lệnh cấu trúc như ***\hline, \midrule, \toprule***, ký hiệu xuống dòng \\ và ký tự phân tách cột & được giữ nguyên. Ngược lại, khi các lệnh trên xuất hiện ngoài ngữ cảnh bảng, chẳng hạn trong hình ảnh hoặc câu văn, parser sẽ

loại bỏ chúng để tránh gây nhiễu cho quá trình phân tích nội dung.

- Các công thức toán inline như ***$...$, \(...\)*** và môi trường ***\begin{math}...\end{math}*** được chuyển về dạng ***$...$***, trong khi các công thức dạng khối như ***$$...$$, \[...\], \begin{equation}...\end{equation},\begin{displaymath}...\end{displaymath}*** được chuẩn hóa về môi trường ***\begin{equation} ... \end{equation}***. Các lệnh định dạng bên trong công thức như ***\mathbf{} hoặc \mathrm{}*** được unwrap để giữ lại ký hiệu toán học gốc. Đối với các environment toán học lồng nhau, chẳng hạn như ***\begin{aligned} ... \end{aligned}*** nằm bên trong một khối equation, parser thực hiện tách thành nhiều equations độc lập để dễ xử lý và trích xuất.
- Parser thực hiện trích xuất tài liệu tham khảo bằng cách tìm các ***\bibitem***. Quá trình gồm các công đoạn chính:
    - Trước hết, parser sử dụng cơ chế pattern matching để phát hiện các dạng khai báo ***\bibitem***, bao gồm cả trường hợp có tham số tùy chọn và không có tham số tùy chọn, chẳng hạn như ***\bibitem[optional]{key}*** hoặc ***\bibitem{key}.*** Nội dung văn bản đi kèm mỗi \bibitem sau đó được thu thập làm đầu vào cho bước phân tích chi tiết.
    - Tiếp theo, hệ thống thực hiện tách trường thông tin bằng cách kết hợp các biểu thức chính quy và các heuristic. Từ nội dung trích dẫn thô, parser cố gắng trích xuất các trường quan trọng như tên tác giả, tiêu

đề bài báo, nơi xuất bản năm xuất bản, số tập và số trang. Do cách trình bày tài liệu tham khảo trong LaTeX không đồng nhất giữa các bài báo, quá trình trích xuất được thiết kế linh hoạt để xử lý nhiều biến thể khác nhau của cùng một cấu trúc tham khảo.

- Sau khi các trường thông tin được trích xuất, hệ thống tiến hành chuẩn hóa loại BibTeX entry dựa trên heuristic nội dung các trường dữ liệu. Các kiểu entry không chuẩn, lỗi chính tả hoặc các biến thể khác nhau được ánh xạ về các loại chuẩn của BibTeX như article, inproceedings, book hoặc misc.

- Sau giai đoạn này, hệ thống thu được cấu trúc phân cấp nội dung bài báo đã được chuẩn hóa và tập tài liệu tham khảo ở định dạng BibTeX chuẩn.

3.3.2. Loại bỏ trùng lặp

- Parser xử lý các trùng lặp tài liệu tham khảo theo bốn trường hợp chính. Trường hợp thứ nhất là khi các entry có cùng citation key và cùng nội dung, trong đó các entry dư thừa sẽ bị loại bỏ. Trường hợp thứ hai xảy ra khi các entry có cùng citation key nhưng nội dung chỉ tương tự nhau ở mức cao (> 85%); khi đó, parser tiến hành hợp nhất các entry bằng cách chuẩn hóa các trường thông tin và ưu tiên giữ lại giá trị đầy đủ hoặc dài hơn cho mỗi trường. Trường hợp thứ ba là khi các entry có citation key khác nhau nhưng nội dung giống nhau; trong tình huống này, parser chọn một key làm chuẩn, đồng thời cập nhật

lại các lệnh trích dẫn trong cấu trúc hierarchy.json để đảm bảo tất cả các citation đều trỏ về key đã được chuẩn hóa. Trường hợp cuối cùng là khi các entry vừa có key khác nhau vừa có nội dung tương tự nhau; khi đó, parser kết hợp cả hai thao tác hợp nhất các trường và cập nhật citation key. Mức độ tương đồng giữa các entry được tính toán dựa trên các trường quan trọng như tiêu đề, tác giả và năm xuất bản.

- Với papers có nhiều file main tex trong version 1, các document nodes được đánh số thứ tự (Document 01, Document 02, v.v.). Các document nodes thuộc phiên bản thứ hai trở đi được so khớp với document của phiên bản đầu tiên bằng cách sử dụng Overlap Coefficient dựa trên nội dung các phần tử con. Khi hai document được xác định là tương ứng, các phần tử con của document ở phiên bản sau sẽ được gán lại (reparent) vào document tương ứng của phiên bản đầu tiên, trong khi document dư thừa sẽ bị loại bỏ. Cuối cùng, parser loại trùng các phần tử có nội dung hoàn toàn giống nhau.
- Sau khi hoàn tất giai đoạn loại bỏ trùng lặp, hệ thống tạo ra các đầu ra đã được làm sạch. Đối với tài liệu tham khảo, tập BibTeX entries không còn trùng lặp, mỗi tài liệu được biểu diễn bằng một citation key duy nhất với các trường thông tin đầy đủ và nhất quán. Đối với nội dung bài báo, cấu trúc phân cấp trong hierarchy.json đã được hợp nhất giữa các phiên bản, loại bỏ các document và

phần tử dư thừa, đồng thời đảm bảo mỗi thành phần nội dung chỉ xuất hiện một lần trong cấu trúc cuối cùng.

3.3.3. Các chỉ số thống kê

- Trung bình inline math/paper: 1164.58
- Trung bình block math/paper: 69.89
- Trung bình formatting removed/paper: 393.93
- Tổng papers có bibitem: 1602
- Tổng số bibitem: 65575
- Trung bình bibitem/paper: 40.933208489388264
- Trung bình bibitem phải dedup/paper: 4.557623049219688
- Trung bình text phải dedup/paper: 169.33648559489774

4. Quy trình so khớp tài liệu tham khảo

Mục tiêu của quy trình so khớp tài liệu tham khảo là tự động xác định mối liên hệ giữa các tài liệu tham khảo được trích xuất từ mã nguồn LaTeX của bài báo và các bản ghi metadata tương ứng từ Semantic Scholar được định danh bằng arXiv ID. Quy trình này nhằm giải quyết bài toán liên kết thực thể nếu ở tương lai tác giả muốn thay đổi trường dữ liệu ở BibTex entries hay arXiv ID.

4.1. Chọn dữ liệu

- Chọn 5 bài báo sẽ được gán nhãn thủ công bao gồm các bài 2411-10224, 10231, 10232, 10261, 10281 thỏa yêu

cầu trong refs.bib chứa tối thiểu 20 BibTeX và trong references.json chứa tối thiểu 20 arXivID.

- Đối với bài báo được gán nhãn tự động, refs.bib có ít nhất 15 BibTeX entries và tệp references.json tương ứng phải chứa tối thiểu 15 arXiv ID đóng vai trò là các reference candidates. Lấy tối đa 3000 bài báo thỏa các tiêu chí trên và các bài báo này không trùng với 5 bài báo thủ công.
- Kết quả có tổng 636 bài báo (5 bài thủ công và 631 bài tự động) thỏa các yêu cầu trên và có tối thiểu 15 số trùng khớp tài liệu tham khảo giữa hai bên.

4.2. Tiền xử lý dữ liệu

4.2.1. Tiêu đề

Ý tưởng: Tiêu đề trong BibTeX thường chứa LaTeX định dạng như {} để giữ chữ viết hoa, các lệnh \textbf{}, \textit{}, \emph{} để in đậm/nghiêng, và ký tự escape như \&, \%. Tiêu đề từ Semantic Scholar có thể chứa MathML tags và HTML entities. Parser xóa tất cả định dạng này, chuyển về lowercase, xóa dấu câu, và chuẩn hóa khoảng trắng để hai tiêu đề giống nhau về ngữ nghĩa sẽ so khớp được.

Xử lý BibTeX:

1. Xóa dấu ngoặc nhọn xung quanh nội dung: {Deep} → Deep
2. Bỏ các lệnh định dạng: \textbf{NLP} → NLP, \emph{model} → model

3. Thay ký tự escape: \& → &, \% → %, _ → _
4. Xóa các lệnh LaTeX còn lại: \\command → ``
5. Chuẩn hóa: chuyển về chữ thường + xóa dấu câu + bỏ khoảng trắng

Xử lý Semantic Scholar:

1. Xóa MathML tags: <mml:math>...</mml:math> → ``
2. Xóa HTML tags còn lại: ₂ → ``
3. Thay HTML entities: & → &, < → <, > → >
4. Chuẩn hóa: chuyển về chữ thường + xóa dấu câu + chuẩn hóa khoảng trắng

Ví dụ:

- **BibTeX:** "{Deep} Learning for \textbf{NLP} \& Applications" → "deep learning for nlp applications"
- **Semantic Scholar:** "Deep Learning & NLP" → "deep learning nlp"

4.2.2. Tác giả

Ý tưởng: Tên tác giả trong BibTeX thường theo định dạng "LastName, FirstName" và phân cách bằng "and". Tên có thể chứa ký tự Unicode (é, ü, ñ). Parser đảo ngược định dạng về "FirstName LastName", dùng unidecode chuyển ký tự có dấu thành ASCII, và chuẩn hóa để tên tác giả từ các nguồn khác nhau có thể so khớp.

Xử lý:

1. Chia danh sách tác giả theo and: "A and B and C" → ["A", "B", "C"]
2. Với mỗi tên:
    - Xóa dấu ngoặc nhọn: {Team Name} → Team Name
    - Đảo format nếu có dấu phẩy: "Nguyễn, Văn A." → "Văn A. Nguyễn"
    - Unicode: "văn" → "van", "Nguyễn" → "Nguyen"
    - Chuẩn hóa: lowercase + xóa dấu câu + chuẩn hóa khoảng trắng
3. Kết hợp các tên tác giả lại bằng dấu “,”.

Ví dụ:

- **Input:** "Nguyễn, Văn A. and Smith, John B. and {TensorFlow Team}"
- **Output:** "van a nguyen, john b smith, tensorflow team"

4.2.3. Năm xuất bản

Ý tưởng: Năm có thể xuất hiện ở nhiều định dạng: "2024", "2024-01-15", "January 2024". Parser dùng regex tìm 4 chữ số liên tiếp đầu tiên làm năm.

Xử lý:

1. Tìm pattern (\d{4}) trong chuỗi

2. Trả về so khớp đầu tiên

Ví dụ:

- **Input:** "2024" → **Output:** "2024"
- **Input:** "2024-01-15" → **Output:** "2024"
- **Input:** "January 15, 2024" → **Output:** "2024"

4.2.4. Nơi xuất bản

Ý tưởng: Trong BibTeX có thể nằm ở các trường khác nhau (journal, booktitle, venue). Parser ưu tiên theo thứ tự này, xóa định dạng và chuẩn hóa để so sánh.

Xử lý BibTeX:

1. Lấy giá trị theo thứ tự ưu tiên: journal → booktitle → venue
2. Xóa dấu ngoặc nhọn và các kí tự escape
3. Chuẩn hóa: chuyển về chữ thường + xóa dấu câu + chuẩn hóa khoảng trắng

Xử lý Semantic Scholar:

1. Lấy trường venue
2. Chuẩn hóa: chuyển về chữ thường + xóa dấu câu + chuẩn hóa khoảng trắng

Ví dụ:

- **BibTeX:** "Proceedings of the {ACL} \& {EMNLP} 2024" → "proceedings of the acl emnlp 2024"

- **Semantic Scholar:** "Annual Meeting of the ACL" → "annual meeting of the acl"

Kết quả của giai đoạn này là hai tập dữ liệu đã được làm sạch và chuẩn hóa: tập BibTeX entries chuẩn hóa và tập reference candidates chuẩn hóa. Các tập dữ liệu này có cấu trúc rõ ràng, nhất quán về mặt biểu diễn và sẵn sàng để sử dụng trong các bước gán nhãn dữ liệu.

4.3. Gán nhãn dữ liệu

4.3.1. Ý tưởng

- Trước khi thực hiện so sánh, toàn bộ văn bản đầu vào được tiền xử lý nhằm giảm nhiễu. Cụ thể, chuỗi được chuyển về chữ thường, loại bỏ các mạo từ ở đầu câu như “A”, “An”, “The”, xóa dấu câu và rút gọn khoảng trắng. Bước tiền xử lý này giúp các phép đo similarity phản ánh đúng hơn mức độ tương đồng về mặt nội dung.
- Để xác định mức độ tương đồng giữa một BibTeX entry và các reference candidates, parser sử dụng cách tiếp cận dựa trên độ tương đồng chuỗi kết hợp hai phương pháp bổ trợ cho nhau. Phương pháp thứ nhất là SequenceMatcher, chiếm trọng số 60%, nhằm so sánh các chuỗi ký tự liên tục và đặc biệt hiệu quả trong trường hợp tiêu đề chỉ khác nhau bởi việc thêm hoặc bớt một vài từ. Phương pháp thứ hai là Jaccard Similarity, chiếm trọng số 40%, dựa trên so sánh tập hợp từ, phù hợp khi thứ tự từ trong chuỗi khác nhau nhưng nội dung ngữ nghĩa vẫn

tương đương. Điểm tương đồng tổng hợp được tính theo công thức: **combined_score = 0.6 × SequenceMatcher + 0.4 × Jaccard.**

- Một cặp BibTeX entry và reference candidate được coi là khớp nếu đồng thời thỏa mãn các điều kiện sau: độ tương đồng tiêu đề không nhỏ hơn 0.7, độ tương đồng tác giả không nhỏ hơn 0.7, và năm xuất bản cho phép sai lệch trong phạm vi ±1 năm. Với mỗi BibTeX entry, parser duyệt qua toàn bộ tập reference candidates, tính điểm tương đồng cho cả tiêu đề và tác giả, sau đó chỉ giữ lại các cặp thỏa mãn điều kiện. Trong số các cặp hợp lệ này, cặp có giá trị min(title_score, author_score) lớn nhất sẽ được chọn làm kết quả so khớp cuối cùng.
- Groundtruth sau khi gán nhãn sẽ chứa mapping {bib_key: arXiv_id}:

```
{
  "bib_key_1": "arxiv_id_1",
  "bib_key_2": "arxiv_id_2",
  ...
}
```

- Ví dụ:

BibTeX entry:

- key: "smith2023deep"
- title_clean: "deep learning for natural language processing"
- author_clean: "john smith, mary jones"

- year: "2023"

Reference candidate:

- arxiv_id: "2301.12345"
- title_clean: "deep learning for nlp"
- author_clean: "j smith, m jones"
- year: "2023"

Kết quả:

- Độ tương đồng tiêu đề là 0.75
- Độ tương đồng tác giả là 0.72
- author_clean: "j smith, m jones"

➔ Do cả hai giá trị đều vượt ngưỡng quy định và năm xuất bản trùng khớp, cặp này được xác định là so khớp thành công.

- Kết quả đầu ra của giai đoạn này được biểu diễn dưới dạng một cấu trúc ánh xạ, trong đó mỗi khóa BibTeX được liên kết với đúng một arXiv ID tương ứng. Tập nhãn này đóng vai trò phục vụ cho các bước tiếp theo như xây dựng tập huấn luyện và đánh giá mô hình.

4.3.2. Các chỉ số thống kê

- Tổng cặp nhãn: 1906335
- Số nhãn khớp: 19092
- Số nhãn không khớp: 1887243
- Tỉ lệ nhãn khớp: 1.00%

4.4. Xây dựng đặc trưng

- Bài toán được xác định là phân loại nhị phân. Cụ thể, với một cặp (BibTeX Entry, Candidate Reference), mô hình cần dự đoán xác suất chúng là cùng một bài báo (Match = 1) hoặc khác nhau (Non-match = 0).
- Dựa trên định nghĩa trên, đầu vào cho quá trình xây dựng các đặc trưng là những cặp Entry – Reference. Điều này tạo ra một tập dữ liệu dạng bảng, trong đó mỗi dòng đại diện cho một cặp (bib_entry, candidate), và các cột là các đặc trưng mô tả độ tương đồng giữa cặp đó.
- Căn cứ vào dữ liệu BibTeX và Semantic Scholar, ba trường dữ liệu (tiêu đề bài báo, danh sách tác giả, năm xuất bản) hầu như xuất hiện thường xuyên và có thể được xem là định danh của một bài báo. Do đó, các đặc trưng được xây dựng dựa trên ba trường dữ liệu này.

4.4.1. Các đặc trưng liên quan đến tiêu đề

- Bao gồm 7 đặc trưng: title_seq_sim, title_jaccard, title_levenshtein, title_tfidf_sim, title_ngram_overlap, title_trigram_overlap, title_len_ratio.
- Ý tưởng: Tiêu đề là thành phần định danh mạnh nhất của một bài báo. Các cặp bài báo khớp nhau sẽ có tiêu đề gần như giống hệt nhau, dẫn đến điểm similarity tiệm cận 1. Ngược lại, các cặp không khớp sẽ có điểm thấp. Ta sử dụng nhiều phương pháp đo similarity khác nhau để quan sát các khía cạnh khác nhau: độ giống nhau theo chuỗi từ

cho chuỗi ký tự, Jaccard cho tập từ, TF-IDF cho vector trọng số, N-gram cho cấu trúc cụm từ.

- Đặc tả các đặc trưng:

  + title_seq_sim:

    - Ý tưởng: So sánh hai chuỗi tiêu đề để tìm các đoạn ký tự giống nhau liên tiếp.
    - Cách tạo: Chuyển cả hai tiều đề về chữ thường, sau đó tìm chuỗi con chung dài nhất. Tính tỉ lệ = (2 × độ dài chuỗi con chung) / (tổng độ dài hai tiêu đề)
    - Output: Giá trị trong [0, 1]

  + title_jaccard:

    - Ý tưởng: So sánh hai tiêu đề dựa trên tập hợp từ, không quan tâm thứ tự.
    - Cách tạo: Tách mỗi tiêu đề thành danh sách các từ. Đếm số từ xuất hiện ở cả hai tiêu đề (intersection), đếm tổng số từ khác nhau ở cả hai (union). Tính tỉ lệ = intersection / union
    - Output: Giá trị trong [0, 1]

  + title_levenshtein:

    - Ý tưởng: Đo khoảng cách chỉnh sửa - cần bao nhiêu thao tác để biến tiêu đề này thành tiêu đề kia.

- Cách tạo: Đếm số thao tác tối thiểu gồm: thêm ký tự, xóa ký tự, thay thế ký tự. Chuẩn hóa bằng cách lấy 1 trừ đi (số thao tác / độ dài title dài hơn)
- Output: Giá trị trong [0, 1]

+ title_tfidf_sim:

- Ý tưởng: Từ xuất hiện nhiều trong corpus (như "the", "of") ít quan trọng, từ hiếm (như tên riêng) quan trọng hơn.
- Cách tạo: Với mỗi từ trong tiêu đề, tính trọng số = (số lần xuất hiện trong title) × log(tổng số tài liệu / số tài liệu chứa từ đó). Biểu diễn mỗi tiêu đề = thành vector trọng số. Tính cosine góc giữa hai vector
- Output: Cosine similarity trong [0, 1]

+ title_ngram_overlap:

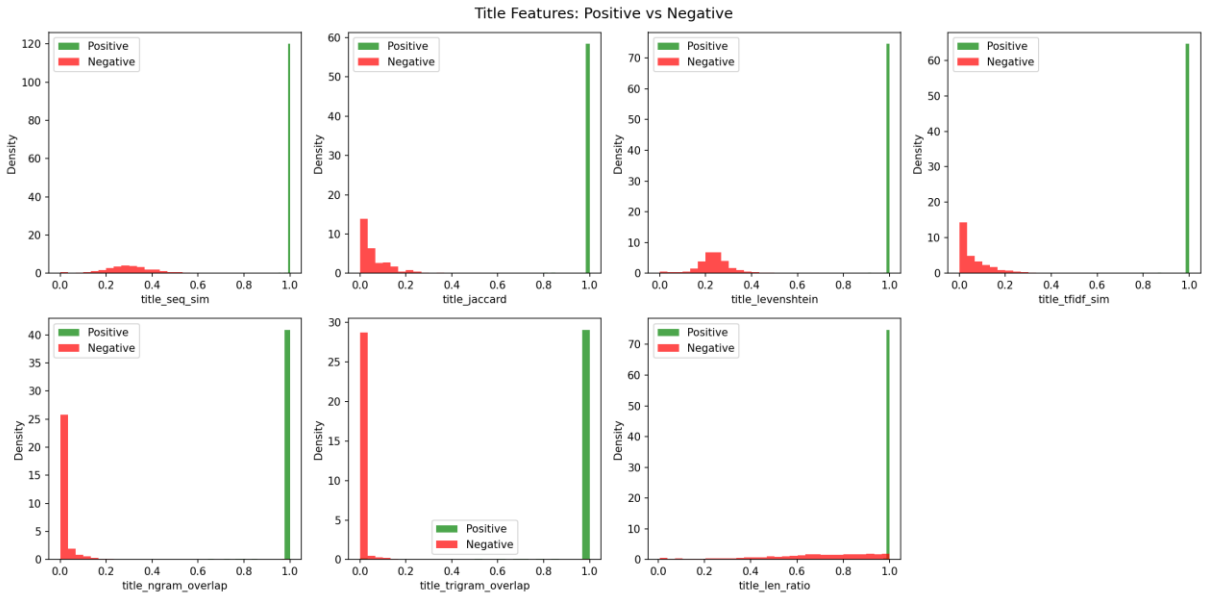
- Ý tưởng: So sánh cụm 2 từ liên tiếp thay vì từ đơn lẻ để hiểu ngữ cảnh.
- Cách tạo: Tách tiêu đề thành các cặp từ liên tiếp. Ví dụ: "deep learning for nlp" → {"deep learning", "learning for", "for nlp"}. Tính Jaccard giữa hai tập bi-grams, Jaccard = số từ xuất hiện ở cả 2 tập / tổng số từ khác nhau của 2 tập
- Output: Giá trị trong [0, 1]

+ title_trigram_overlap:

- Ý tưởng: Tương tự bi-gram nhưng với cụm 3 từ, chặt chẽ hơn.
- Cách tạo: Tách tiêu đề thành các bộ 3 từ liên tiếp. Ví dụ: "deep learning for nlp" → {"deep learning for", "learning for nlp"}. Tính Jaccard = số từ xuất hiện ở cả 2 tập / tổng số từ khác nhau của 2 tập
- Output: Giá trị trong [0, 1]

+ title_len_ratio:

- Ý tưởng: Hai tiêu đề cùng bài báo thường có độ dài tương đương.
- Cách tạo: Đếm số ký tự mỗi tiều đề. Chia độ dài tiêu đề ngắn hơn cho độ dài tiêu đề dài hơn
- Output: Giá trị trong [0, 1]



Đặc trưng	**Positive trung bình**	**Negative trung bình**	**Nhận xét**
title_seq_sim	0.998	0.302	Phân tách rất rõ – Positive gần 1, Negative ~0.3
title_jaccard	0.994	0.062	Phân tách xuất sắc – Negative gần 0
title_levenshtein	0.997	0.241	Tương tự seq_sim
title_tfidf_sim	0.995	0.065	Tương tự jaccard
title_ngram_overlap	0.992	0.016	Phân tách tốt nhất
title_trigram_overlap	0.986	0.007	Gần như hoàn hảo
title_len_ratio	0.997	0.666	Phân tách yếu hơn

Kết luận: Các đặc trưng liên quan tiêu đề phân tách rất tốt giữa Positive và Negative. Positive luôn gần 1, Negative gần 0.

4.4.2 Các đặc trưng liên quan đến tác giả

- Bao gồm 5 đặc trưng: author_jaccard, author_tfidf_sim, first_author_match , num_common_authors, author_count_diff.
- Ý tưởng: Danh sách tác giả là yếu tố định danh quan trọng thứ hai sau tiêu đề. Tuy nhiên, tên tác giả thường bị định dạng khác nhau giữa các nguồn: "Nguyễn Văn A" có thể được viết là "V.A. Nguyen" hoặc "Nguyen, Van A.". Vì vậy, ta sử dụng nhiều phương pháp khác nhau: Jaccard để so sánh tập tên, TF-IDF để ưu tiên tên hiếm, first author match để kiểm tra tác giả chính, và đếm số tác giả trùng để đảm bảo số lượng tác giả tương đương.
- Đặc tả các đặc trưng:

  + author_jaccard:

    - Ý tưởng: So sánh hai danh sách tác giả dựa trên tập hợp tên, không quan tâm thứ tự.
    - Cách tạo: Tách chuỗi tác giả thành danh sách các từ (tên, họ). Đếm số từ xuất hiện ở cả hai danh sách (phần giao), đếm tổng số từ khác nhau ở cả hai (phần hợp). Tính tỉ lệ = phần giao / phần hợp
    - Output: Giá trị trong [0, 1]

  + author_tfidf_sim:

- Ý tưởng: Tên tác giả hiếm (như họ đặc biệt) quan trọng hơn tên phổ biến (như "John", "Smith")
- Cách tạo: Với mỗi từ trong danh sách tác giả, tính trọng số dựa trên tần suất xuất hiện trong toàn bộ danh sách. Tên hiếm có trọng số cao. Biểu diễn mỗi danh sách tác giả thành vector trọng số, tính cosine góc giữa hai vector.
- Output: Cosine similarity trong [0, 1]

+ first_author_match:

- Ý tưởng: Tác giả đầu tiên thường là tác giả chính, đóng vai trò quan trọng nhất trong việc định danh bài báo
- Cách tạo: Lấy tác giả đầu tiên từ mỗi danh sách (phần tử trước dấu phẩy đầu tiên). Lấy từ cuối cùng làm họ. So sánh hai họ xem có giống nhau không
- Output: 1 nếu họ của tác giả đầu giống nhau, 0 nếu khác

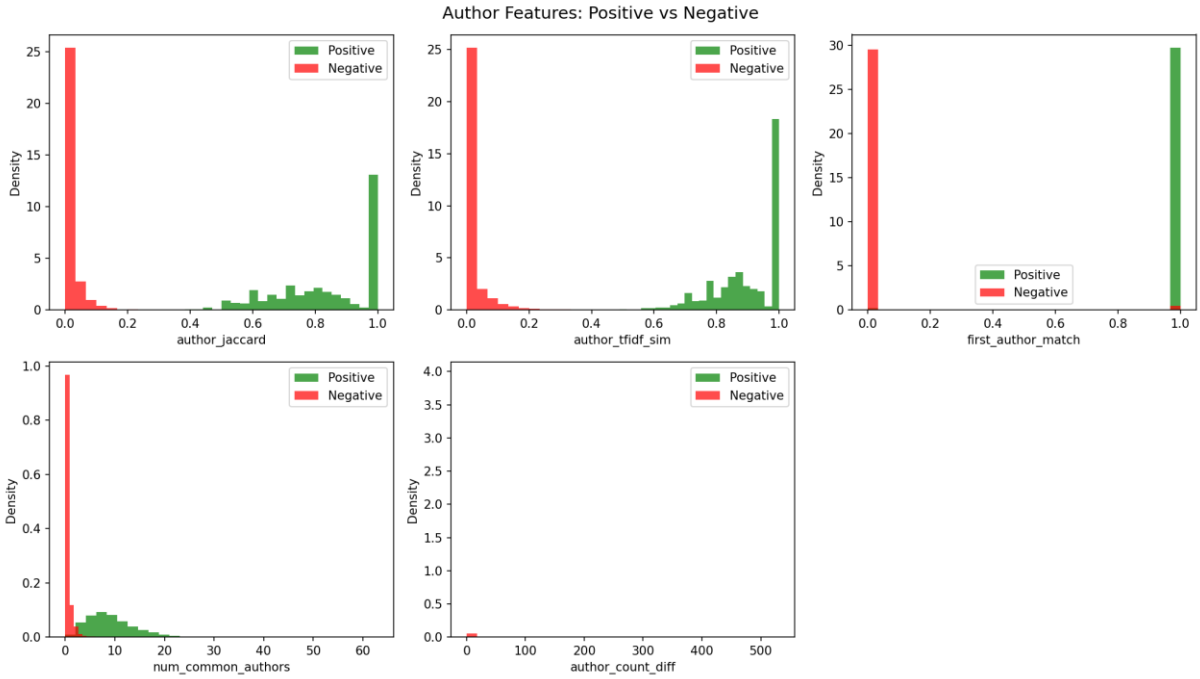
+ num_common_authors:

- Ý tưởng: Cùng một bài báo phải có nhiều tên tác giả trùng nhau.
- Cách tạo: Tách danh sách tác giả thành các từ riêng lẻ. Loại bỏ các từ ngắn hơn 2 ký tự (như initials "J", "A"). Đếm số từ xuất hiện ở cả hai danh sách

- Output: Số nguyên ≥ 0

+ author_count_diff:

- Ý tưởng: Cùng một bài báo phải có số tác giả bằng nhau hoặc gần bằng nhau.
- Cách tạo: Đếm số tác giả trong mỗi danh sách (đếm số dấu phẩy + 1). Tính trị tuyệt đối của hiệu hai số đó
- Output: Số nguyên ≥ 0. Giá trị 0 = số tác giả bằng nhau.



Đặc trưng	**Positive trung bình**	**Negative trung bình**	**Nhận xét**
author_jaccard	0.833	0.014	Phân tách tốt
author_tfidf_sim	0.900	0.019	Phân tách tốt
first_author_match	0.992	0.015	Phân tách tốt
num_common_authors	9.33	0.30	Positive có trung bình 9 tên chung
author_count_diff	0.11	6.91	Negative lệch nhiều tác giả hơn

Kết luận: Các đặc trưng liên quan tác giả cũng phân tách tốt, đặc biệt first_author_match.

4.4.3. Các đặc trưng liên quan đến năm xuất bản

- Bao gồm 2 đặc trưng: year_diff, year_exact_match
- Ý tưởng: Năm xuất bản là thông tin xác thực quan trọng để loại trừ các false positives. Có thể xảy ra trường hợp hai bài báo khác nhau nhưng có tiêu đề tương tự. Trong trường hợp này, năm xuất bản giúp phân biệt: nếu một bài từ 2020 và một bài từ 2023, chắc chắn không phải cùng bài báo. Tuy nhiên, cần cho phép tolerance ±1 năm vì

cùng một bài có thể xuất hiện dạng preprint (arXiv) trước khi được xuất bản chính thức.

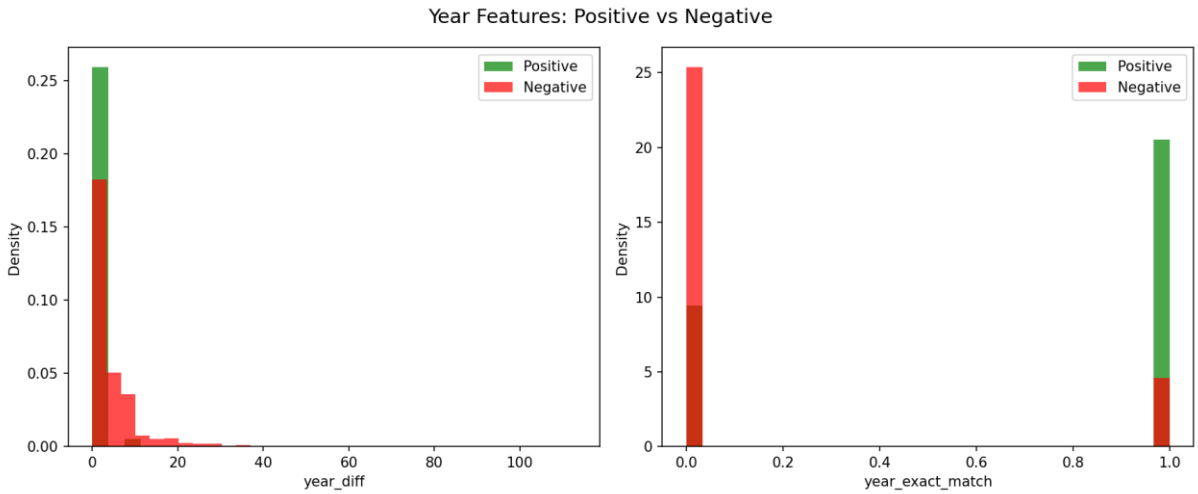
- Đặc tả các đặc trưng:

+ year_diff:

  - Ý tưởng: Cùng một bài báo phải được xuất bản cùng năm hoặc chênh lệch tối đa 1 năm.
  - Cách tạo: Trích xuất năm (4 chữ số). Tính trị tuyệt đối của hiệu hai năm. Nếu thiếu năm ở một trong hai, gán giá trị mặc định cao (10) để đánh dấu là không đáng tin cậy
  - Output: Số nguyên ≥ 0. Giá trị 0 = cùng năm, 1 = chênh 1 năm, >2 = khả năng cao không match.

+ year_exact_match:

  - Ý tưởng: Tạo một đặc trưng nhị phân để model học năm có khớp chính xác hay không
  - Cách tạo: So sánh năm. Nếu cả hai đều có năm và năm bằng nhau → 1, ngược lại → 0
  - Output: 1 nếu năm khớp chính xác, 0 nếu không khớp hoặc thiếu dữ liệu



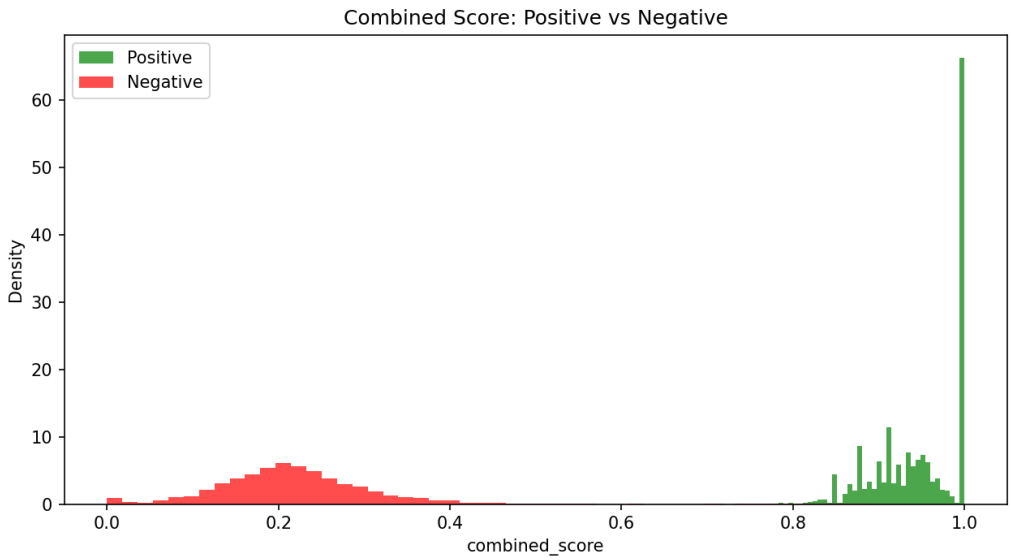
Đặc trưng	Positive trung bình	Negative trung bình	Nhận xét
year_diff	0.55	4.74	Positive chênh ~0.5 năm, Negative chênh ~5 năm
year_exact_match	0.68	0.15	68% positive có năm khớp chính xác

Kết luận: Các đặc trưng có liên quan đến năm xuất bản có phân tách nhưng không rõ ràng như tiêu đề/tác giả.

4.4.4. Đặc trưng tổng hợp:

combined_scores:

- Ý tưởng: Kết hợp thông tin từ tiêu đề và tác giả thành một điểm duy nhất với trọng số phản ánh độ quan trọng
- Cách tạo: Tính weighted sum = 0.7 × title_seq_sim + 0.3 × author_jaccard. Title được cho trọng số cao hơn (70%) vì là yếu tố định danh quan trọng nhất.
- Output: Giá trị trong [0, 1]



Đặc trưng	Positive trung bình	Negative trung bình
combined_score	0.949	0.216

Kết luận: combined_score tổng hợp tốt, phân tách rõ ràng giữa 2 lớp.

4.5. Mô hình hóa dữ liệu

4.5.1. Dữ liệu đầu vào

- Mô hình lựa chọn là phân loại nhị phân. Trong đó, với dữ liệu đầu vào là các cặp (BibTeX Entry, Candidate Reference), mô hình cần dự đoán xác suất chúng là cùng một bài báo (Match = 1) hoặc khác nhau (Non-match = 0).
- Mỗi bài báo có: m BibTeX entries trong file .bib, n reference candidates trong file references.json
- Với mỗi bài báo, tạo m × n cặp (bib_entry, candidate). Mỗi cặp là một data point với 15 features mô tả độ tương đồng giữa bib_entry và candidate, 1 label nhị phân: yes (1) nếu cặp này match, no (0) nếu không match

Ví dụ:

paper_id	**bib_key**	**candidate_id**	**title_seq_sim**	**...**	**label**
2411-10224	smith2023	2301.12345	0.85	...	1
2411-10224	smith2023	2302.54321	0.12	...	0
...	...	...	...	...	...

- Đặc điểm dữ liệu: không cân bằng vì mỗi bib_entry chỉ match với 1 candidate → tỉ lệ positive rất thấp
- Dữ liệu được chia như sau:

Tập	Thành phần	Mục đích
Test	1 bài báo thủ công + 1 bài báo tự động (2 bài báo, 8721 cặp)	Đánh giá cuối cùng
Valid	1 bài báo thủ công + 1 bài báo tự động (2 bài báo, 2322 cặp)	Tinh chỉnh siêu tham số, kĩ thuật dừng sớm
Train	Tất cả bài báo còn lại (632 bài báo, 1895292 cặp)	Huấn luyện mô hình

4.5.2. Mô hình sử dụng

- Trước hết, huấn luyện mô hình baseline XGBoost, Naïve Bayes, và Logistic Regression trên 100 bài báo bất kì của tập dữ liệu thì ta có kết quả như sau:

Model	F1	Accuracy	Precision	Recall	MRR
XG Boost	0.9807	0.9996	0.9651	0.9969	0.5248
Naive Bayes	0.9094	0.9978	0.8349	0.9984	0.5243

Model	F1	Accuracy	Precision	Recall	MRR
Logistic Regression	0.9287	0.9983	0.8668	1.0000	0.5248

- XGBoost được chọn vì đạt F1-score, precision, và MRR cao nhất trong khi vẫn giữ recall gần tuyệt đối, phù hợp nhất cho bài toán reference matching với dữ liệu mất cân bằng.
- Ngoài ra, XGBoost được lựa chọn vì có khả năng xử lý tốt dữ liệu mất cân bằng thông qua tham số scale_pos_weight để cân bằng lớp, không cần chuẩn hóa dữ liệu do là mô hình dựa trên cây, cung cấp sẵn độ quan trọng của từng đặc trưng để phân tích, hỗ trợ cơ chế early stopping giúp tránh overfitting và đạt hiệu suất cao khi làm việc với dữ liệu dạng bảng

4.5.3. Mô tả chi tiết mô hình

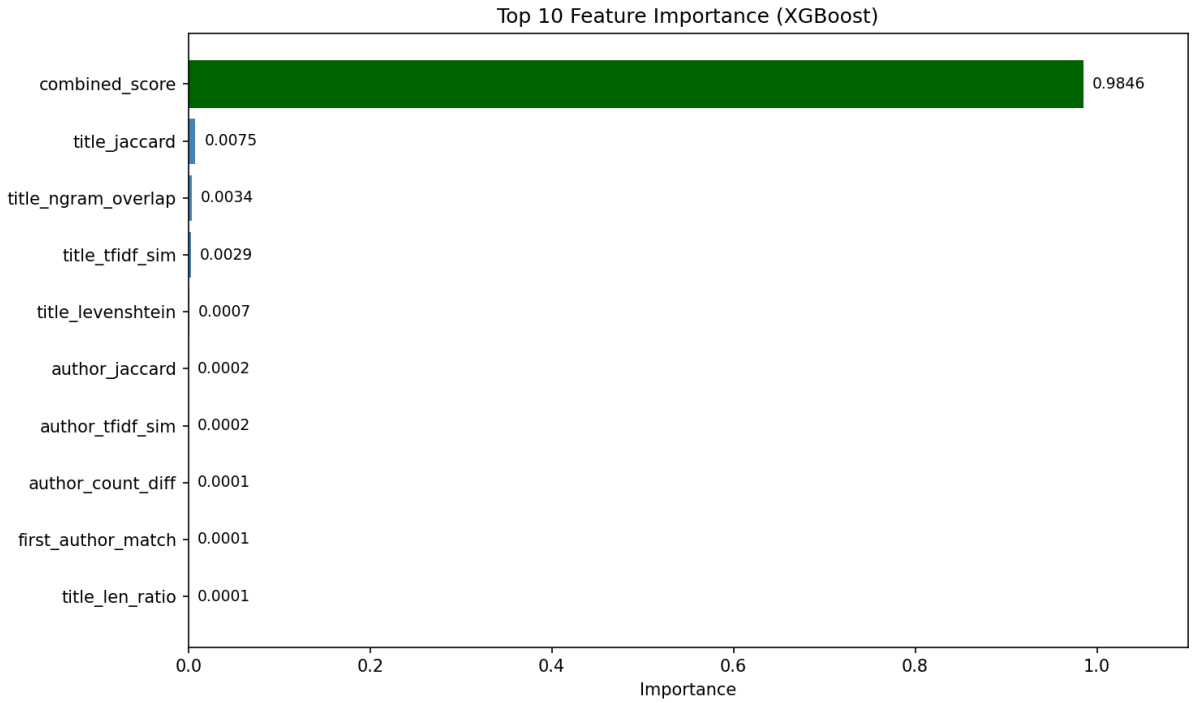
- Cấu trúc:
  - Sự kết hợp của nhiều cây quyết định
  - Mỗi cây được huấn luyện để sửa lỗi của những cây trước đó (gradient boosting)
- Xử lý dữ liệu mất cân bằng: scale_pos_weight = số mẫu negative / số mẫu positive = 98.86. Giá trị này tăng trọng số cho lớp positive trong hàm mất mát.
- Các siêu tham số chính để tinh chỉnh:

Parameter	Ý nghĩa	Grid Search
n_estimators	Số lượng cây	[50, 100, 200]
max_depth	Độ sâu tối đa của mỗi cây	[4, 6, 8]
learning_rate	Tốc độ học	[0.05, 0.1, 0.2]

4.5.4. Các bước huấn luyện mô hình

- Bước 1: Chia tập dữ liệu dạng bảng thành ba phần theo các bài báo trong tập train, valid, và test.
- Bước 2: GridSearchCV tìm siêu tham số tốt nhất
  - 3-fold cross-validation trên tập train
  - Thử 27 tổ hợp (3×3×3) của các siêu tham số
  - Đánh giá bộ tham số tốt nhất dựa trên điểm F1
  - Kết quả: bộ tham số tốt nhất là {'learning_rate': 0.2, 'max_depth': 8, 'n_estimators': 200}
- Bước 3: Train lại mô hình với kĩ thuật dừng sớm và với bộ tham số tốt nhất
  - Tăng n_estimators lên 3 lần
  - Dùng tập valid để dừng sớm (patience=20)
  - Dừng khi độ mất mát đánh giá không giảm sau 20 vòng lặp để tránh overfitting
- Bước 4: Đánh giá trên tập valid
  - Tính các chỉ số đánh giá F1 Score, Accuracy. Kết quả ta có: F1 Score = 0.9897, Accuracy = 0.9996

- In ra các đặc trưng quan trọng sau khi huấn luyện



Nhóm đặc trưng liên quan đến tiêu đề chiếm phần lớn, cho thấy tiêu đề là yếu tố quyết định nhất trong việc so khớp tài liệu tham khảo.

- Bước 5: Dự đoán Top-5 candidates
- Với mỗi bib_entry, lấy 5 candidates có probability cao nhất
- Lưu vào pred.json cho mỗi paper

4.6. Đánh giá mô hình

- Tỉ lệ thành công của so khớp tài liệu tham khảo trên 3 tập train, valid, test đều đạt 100%. Cụ thể, chỉ số MRR = 1.0

khi đánh giá trên cả 3 tập cho thấy mô hình có khả năng tổng quát hóa tốt, không xảy ra hiện tượng overfitting.

- Mô hình có khả năng xếp hạng chính xác rất cao, phần lớn đáp án đúng đều nằm ở vị trí đầu tiên (Rank 1).
- Ground truth được tạo bằng matching tự động với threshold cao (title_sim ≥ 0.7, author_sim ≥ 0.7). Do đó, mô hình XGBoost kết hợp với bộ ba nhóm đặc trưng (tiêu đề, tác giả, năm xuất bản) chứng minh rằng việc kết hợp các đặc trưng để huấn luyện mô hình là hiệu quả, có liên quan đến ground truth mặc dù mô hình không học trong quá trình gán nhãn.

5. Link Youtube

- Youtube: https://youtu.be/D0gJiSpljNE
- Google Drive: https://drive.google.com/file/d/1p8kTJ0Ic4tEWDe5aaesusmZvGeblUz7Z/view?usp=sharing

6. Tài liệu tham khảo

Using XGBoost in Python Tutorial , ngày truy cập: 15/01/2026

Learn XGBoost in Python: A Step-by-Step Tutorial | DataCamp

Scan_Bibtex_Entries , ngày truy cập: 15/01/2026

Scan_Bibtex_Entries/scan_bibtex_file.py at main · DLR-SF/Scan_Bibtex_Entries

Slide tài liệu môn học Nhập môn Khoa học dữ liệu – Trường Đại học Khoa học tự nhiên